

Ограничения в БД «Авиаперевозки»

Таблица flights

```
\d flights
```

```
CHECK CONSTRAINTS
```

Практический вопрос:

- Объяснить смысл ограничений-проверок в этой таблице

Ограничения в БД «Авиаперевозки»

Таблица flights

Ограничения-проверки:

Ограничения-проверки:

```
"flight_actual_check" CHECK (actual_arrival IS NULL  
    OR actual_departure IS NOT NULL  
    AND actual_arrival IS NOT NULL  
    AND actual_arrival > actual_departure)
```

```
"flight_scheduled_check" CHECK  
    (scheduled_arrival > scheduled_departure)
```

```
"flight_status_check" CHECK  
    (status = ANY (ARRAY['Scheduled'::text, 'On Time'::text,  
        'Delayed'::text, 'Boarding'::text, 'Departed'::text,  
        'Arrived'::text, 'Cancelled'::text]))
```

5. Уровни изоляции (SQL92), блокировки, MVCC

Аномалия / Уровень	Lost update	Грязное чтение Dirty read	Неповторяемое чтение Nonrepeatable	Фантомное чтение Phantom	Serialization Anomaly
Read uncommitted	Нет	В постгресе — нет, в стандарте - да	Да	Да	Да
Read committed	Нет	Нет	Да	Да	Да
Repeatable read	Нет	Нет	Нет	Да, но не в постгресе	Да
Serializable	Нет	Вообще никаких аномалий!			

Изоляция транзакций: давайте попрактикуемся

Открываем три сессии в БД.

Производим действия в них (SELECT, UPDATE), наблюдая эффекты изоляции.

В третьей сессии - мониторим блокировки с помощью системной таблицы `pg_lock` и с помощью расширения `pgrowlocks`.

Также: `pg_current_snapshot()` и `txid_current()`

- 1) Какие вообще есть уровни изоляции?
- 2) Какие возникают блокировки при создании транзакций и в ходе их выполнения?
- 3) Как это зависит от уровня изоляции?

Read Committed

```
BEGIN;
```

```
SELECT status  
FROM flights WHERE  
flight_id=1185;  
..Cancelled..
```

```
SELECT status  
FROM flights WHERE  
flight_id=1185;  
..Scheduled..
```

```
COMMIT;
```

```
UPDATE flights  
SET status = 'Scheduled'  
WHERE flight_id = 1185;
```

Изоляция транзакций

Repeatable Read

```
BEGIN ISOLATION LEVEL  
    REPEATABLE READ;
```

```
SELECT status  
FROM flights WHERE  
flight_id=1185;  
..Scheduled..
```

```
SELECT status  
FROM flights WHERE  
flight_id=1185;  
..Scheduled..
```

```
COMMIT;
```

```
UPDATE flights  
SET status = 'Cancelled'  
WHERE flight_id = 1185;
```

MVCC

xmin	xmax	id	Контент
800	900	1185	... Cancelled
900		1185	... Scheduled ...

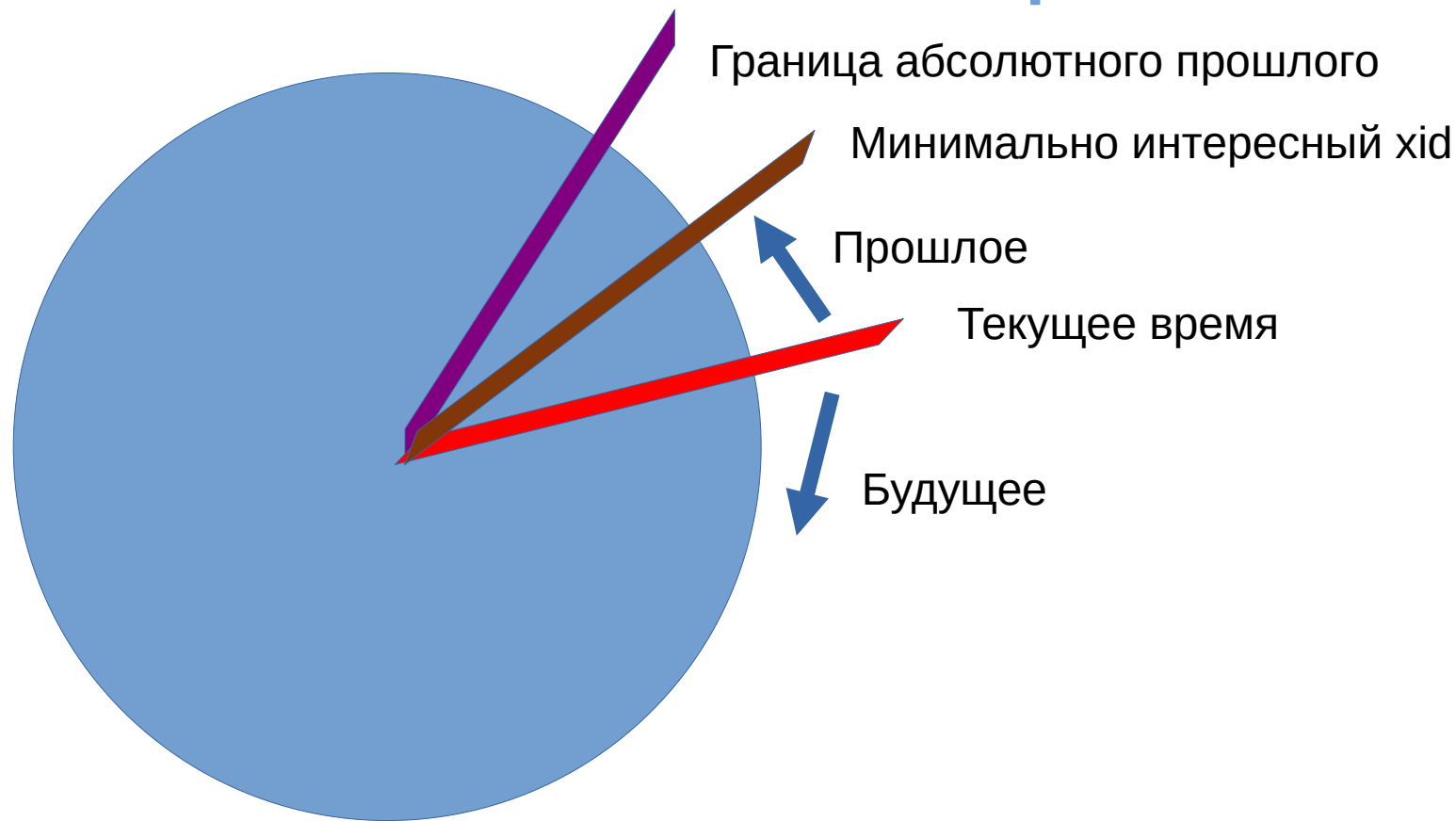
Вот зачем нумеровать транзакции.

Что делать с конечностью времени

xmin	xmax	id	frozen	Контент
800	900	1185		... Cancelled
900		1185		... Scheduled ...
400	500	1184		Это можно удалять
400		1184	Да	Это надо заморозить

Нет ни одной открытой транзакции, начавшейся ранее 600,
Т.е. «Абсолютное прошлое = 600»

VACUUM, wraparound



Нагружаем БД

- Как СУБД пишет данные на «диск»
- Пустим тестовую нагрузку с помощью `pgbench`
- Ещё немного SQL

СУБД и диск

- 1) Реальная работа идет в памяти (buffer cache)
- 2) Асинхронная запись
- 3) Но: сохранность данных (Durability)

КАК ?

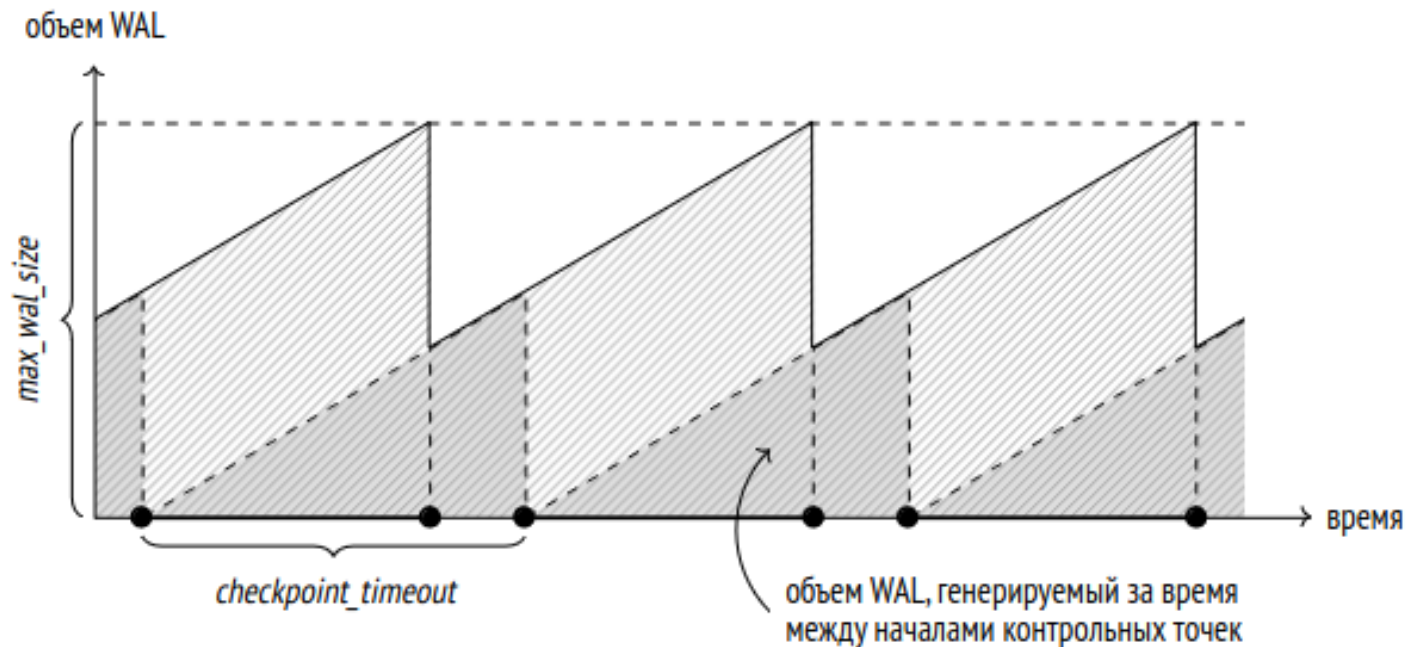
- WAL (Write Ahead Log) — пишется непрерывно
- Сброс буфера (checkpointer, bgwriter, и др) — время от времени

WAL

- Write Ahead Log – журнал предварительной записи
- Состоит из записей, имеющих строгую последовательность. См LSN. Записи есть разных типов. Примерно так:
 - команда для выполнения
 - full page image
- Пишется файлами по 16Mb
- Ненужные удаляются автоматически

Сброс буферов

- checkpointер `SHOW checkpoint_timeout;`
- сбрасывает кэш время от времени



- Bgwriter — помогает ему по ходу загрязнения буферов

Startup recovery

- Определить, насколько БД на диске отстаёт от WAL
- Догнать WAL'ами базу
- Начать работать.

Другие использования WAL

- Репликация базы данных
- Резервное копирование с PITR (восстановление на точку времени)

База и её конфиг-файл

/etc/postgresql/17/main/**postgresql.conf**

```
shared_buffers = 128MB          # min 128kB  
work_mem = 4MB                  # min 64kB
```

.....

База и её пользователи

```
CREATE USER ivanov
```

Пользователь БД ≠ Пользователь ОС. Но ...

```
psql -U ivanov dbname
```

```
sudo -u ivanov psql dbname
```

/etc/postgresql/17/main/**pg_hba.conf**

local	all	all		peer
local	benchdb	ivanov		trust
host	all	all	127.0.0.1/32	scram-sha-256

```
SELECT pg_reload_conf();
```

```
sudo pg_ctlcluster 17 main reload
```

```
sudo service postgresql reload
```

```
sudo systemctl reload postgresql.service
```

pgbench

createdb **benchdb** #тестовая БД

pgbench -i -s 10 **benchdb** # инициализация на 1 млн «аккаунтов»

psql **benchdb** -c "
select pg_size_pretty(pg_database_size ('benchdb'));"
узнать объем

pgbench -c 10 -t 1000 **benchdb**
запуск 10 клиентов по 1000 транзакций

latency average = 6.679 ms
initial connection time = 34.134 ms
tps = 1497.325627 (without initial connection time)

pgbench -c 10 -R 1000 -T 3600 **benchdb**
час гнать 10 клиентов по 1000 TPS

На что смотреть?

- 1) Загрузка системы (top, atop и прочее)
- 2) Логи постгреса `/var/log/postgresql/postgresql-17-main.log`
- 3) Размеры файлов, появление и размер WAL-файлов
- 4) Ситуация может меняться со временем, потому что....
 - «версионник». При UPDATE создаётся новая версия. Происходит «распухание» (bloat).
 - Происходят checkpoint'ы (см далее)
 - Работает VACUUM — борется с распуханием (и не только)
 - и многое другое...

Некоторые запросы

- 1) `SELECT relname, relfrozenxid, age(relfrozenxid)
FROM pg_class WHERE relkind = 'r' ORDER BY age;`
- 2) `SELECT pg_current_wal_insert_lsn();` -- LSN=Log Sequence
Number
- 3) `SELECT count(*) from pg_buffercache where isdirty;`
- 4) `https://wiki.postgresql.org/wiki/Show\_database\_bloat`



КОНЕЦ

К следующему разу:

Установить упоминавшиеся расширения и pageinspect

Что будем делать:

Попробуем загрузить БД

Изучим и попробуем разные типы индексов